



Πανεπιστήμιο Δυτικής Αττικής
Σχολή Μηχανικών
Τμήμα Μηχανικών Πληροφορικής και Υπολογιστών

Διπλωματική Εργασία

Μελέτη και ανάπτυξη τεχνικών για την παρακολούθηση και την
αποσφαλμάτωση της εκτέλεσης εντολών σε υπολογιστικά
συστήματα

Χρήστος Μαργιώλης
Α.Μ. 19390133

Εισηγητής: Παναγιώτης Καρχαζής

Διπλωματική Εργασία

Μελέτη και ανάπτυξη τεχνικών για την παρακολούθηση και την αποσφαλμάτωση της εκτέλεσης εντολών σε υπολογιστικά συστήματα

Χρήστος Μαργιώλης
Α.Μ. 19390133

Εισηγητής:

Παναγιώτης Καρκαζής, i++i

Εξεταστική επιτροπή:

i++i

Ημερομηνία εξέτασης: i++i

Δήλωση συγγραφέα διπλωματικής εργασίας

Ο κάτωθι υπογεγραμμένος **Μαργιώλης Χρήστος** του **Δημητρίου**, με αριθμό μητρώου **19390133**, φοιτητής του Τμήματος Μηχανικών Πληροφορικής και Υπολογιστών της Σχολής Μηχανικών του Πανεπιστημίου Δυτικής Αττικής, δηλώνω υπεύθυνα ότι:

«Βεβαιώνω ότι είμαι συγγραφέας αυτής της Διπλωματικής εργασίας και κάθε βοήθεια την οποία είχα για την προετοιμασία της, είναι πλήρως αναγνωρισμένη και αναφέρεται στην εργασία. Επίσης, οι όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε ακριβώς είτε παραφρασμένες, αναφέρονται στο σύνολό τους, με πλήρη αναφορά στους συγγραφείς, τον εκδοτικό οίκο ή το περιοδικό, συμπεριλαμβανομένων και των πηγών που ενδεχομένως χρησιμοποιήθηκαν από το διαδίκτυο. Επίσης, βεβαιώνω ότι αυτή η εργασία έχει συγγραφεί από μένα αποκλειστικά και αποτελεί προϊόν πνευματικής ιδιοκτησίας τόσο δικής μου, όσο και του Ιδρύματος. Παράβαση της ανωτέρω ακαδημαϊκής μου ευθύνης αποτελεί ουσιώδη λόγο για την ανάκληση του πτυχίου μου».

Ο Δηλών
i++i

Ευχαριστίες

Την προσπάθειά μου αυτή υποστήριξε ο επιβλέπων καθηγητής μου Παναγιώτης Καρκαζής, τον οποίο θα ήθελα να ευχαριστήσω.

Ακόμα θα ήθελα να ευχαριστήσω την οικογένειά μου για τη συμπαράσταση κατά τη διάρκεια των σπουδών μου, καθώς και τους Mark Johnston, Mitchell Horne και το FreeBSD Foundation, για την πολύτιμη βοήθειά τους.

Περίληψη

Η εργασία αποσκοπεί στην μελέτη τεχνικών που χρησιμοποιούνται στην ανάλυση και αποσφαλμάτωση λογισμικού μέσω της καταγραφής και παρακολούθησης των εντολών που εκτελούνται σε ένα επεξεργαστή. Στο πλαίσιο της διπλωματικής θα σχεδιαστεί και θα αναπτυχθεί επέκταση του εργαλείου DTrace η οποία θα παρέχει την δυνατότητα παρακολούθησης οποιασδήποτε μεμονωμένης εντολής assembly εντός μιας δεδομένης συνάρτησης του πυρήνα του λειτουργικού συστήματος FreeBSD.

i++i

Abstract

i++i

Περιεχόμενα

1	Συντομογραφίες	6
2	Εισαγωγή στην παρακολούθηση (tracing)	6
3	Εισαγωγή στο DTrace	6
3.1	Providers	6
3.1.1	FBT	6
3.2	Probes	6
3.3	Γλώσσα D	6
3.3.1	Δομή εντολών DTrace	6
3.3.2	Global μεταβλητές	7
3.3.3	CTF	7
3.3.4	DIF	7
3.4	Επικοινωνία δεδομένων μεταξύ kernel και userspace	7
3.5	Scripts υψηλότερου επιπέδου	7
3.6	Παραδείγματα	7
4	kinst	7
4.1	Χρήση	8
4.2	Γιατί είναι χρήσιμο κάτι τέτοιο;	9
4.3	Ενορχήστρωση εντολών assembly	9
4.4	«Τραμπολίνο»	10
4.4.1	Εσωτερική διάταξη	11
4.4.2	Σημειώσεις υλοποίησης για x86-64	11
4.4.3	Σημειώσεις υλοποίησης για ARM64 και RISC-V	12
4.5	Παρακολούθηση inline συναρτήσεων	13
4.5.1	Τι είναι inline συναρτήσεις	14
4.5.2	Γιατί είναι δύσκολη η παρακολούθηση inline συναρτήσεων	14
4.5.3	Πως λύνει ο kinst αυτό το πρόβλημα	14
4.5.4	Πρότυπο DWARF	14
4.5.5	Πρότυπο ELF	16
4.5.6	Υπολογισμός ορίων κλήσεων	16
4.5.7	Εύρεση καλούσας συνάρτησης	18
4.5.8	Υπολογισμός των offsets entry και return	18
5	Πειράματα	19
6	Συμπεράσματα	19
7	Βιβλιογραφία	20
8	Παράρτημα	21

1 Συντομογραφίες

i++i

2 Εισαγωγή στην παρακολούθηση (tracing)

Με τον όρο tracing εννοούμε την παρακολούθηση της ροής εκτέλεσης ενός προγράμματος σε πραγματικό χρόνο (real-time), με σκοπό την εξαγωγή δεδομένων, που είναι χρήσιμα κυρίως στην αποσφαλμάτωση (debugging) του προγράμματος, στην καλύτερη κατανόηση της ροής εκτέλεσης, καθώς και στην εύρεση τυχόν σημείων που προκαλούν καθυστερήσεις (bottleneck). Κατά μία έννοια το tracing μπορεί να θεωρηθεί ως μία καλύτερη και πιο δυναμική μορφή καταγραφής πληροφοριών (logging), διότι δίνεται η δυνατότητα στον χρήστη να εξάγει αφηρημένες πληροφορίες από ένα πρόγραμμα, χωρίς να χρειάζεται αναγκαστικά η συγγραφή περαιτέρω πηγαίου κώδικα μέσα στο πρόγραμμα και η επαναμεταγλώττισή του, κάτι το οποίο, στην περίπτωση μεγάλων προγραμμάτων όπως ο πυρήνας ενός λειτουργικού συστήματος, τείνει να είναι μία χρονοβόρα διαδικασία.

i++i

3 Εισαγωγή στο DTrace

Το DTrace είναι ένα framework δυναμικής παρακολούθησης (dynamic tracing framework), το οποίο προήλθε από το λειτουργικό σύστημα Solaris το 2005 και έπειτα μεταφέρθηκε στο FreeBSD. Μεταξύ άλλων, χρησιμοποιείται για μετρήσεις επιδόσεων, αποσφαλμάτωση και παρακολούθηση σε πραγματικό χρόνο, καθώς και παρατήρηση συμπεριφοράς συγκεκριμένων υποσυστημάτων του λειτουργικού συστήματος. Το DTrace είναι ιδιαίτερα ισχυρό χάρη στην γλώσσα D (βλέπε ενότητα 3.3) — μία γλώσσα scripting που παρομοιάζει την C και την AWK, η οποία περιέχει μεγάλη γκάμα λειτουργιών και συναρτήσεων και χρησιμοποιείται για την σύνταξη DTrace scripts.

i++i

3.1 Providers

Στην ορολογία του DTrace, ως provider ορίζεται

i++i

3.1.1 FBT

i++i

3.2 Probes

i++i

3.3 Γλώσσα D

i++i

3.3.1 Δομή εντολών DTrace

Οι εντολές DTrace ορίζονται ως εξής:

```
<provider>:<module>:<function>:<name> /<predicate>/ {<actions>}
```

i++;

3.3.2 Global μεταβλητές

i++;

3.3.3 CTF

i++;

3.3.4 DIF

Για να εισαχθεί ένα D script στον πυρήνα του λειτουργικού συστήματος, χρησιμοποιείται η κωδικοποίηση DIF (D Intermediary Format). Η κωδικοποίηση αυτή έχει ως στόχο την μεταγλώττιση του D script σε μία ενδιάμεση byte κωδικοποίηση (byte code), η οποία μπορεί να εκτελεστεί από το DTrace εντός του kernel. Αφού το script έχει μεταγλωττιστεί, φορτώνεται στο kernel από το DTrace, και εκτελείται για τα probes που έχουν οριστεί όταν αυτά ενεργοποιηθούν.

Αντίστοιχα στο Linux, χρησιμοποιείται η κωδικοποίηση eBPF.

i++;

3.4 Επικοινωνία δεδομένων μεταξύ kernel και userspace

Η επικοινωνία δεδομένων μεταξύ kernel και userspace επιτυγχάνεται μέσω ενός ζευγαριού buffers, διαφορετικό για κάθε πυρήνα (per-CPU). Ανά πάσα στιγμή ο ένας buffer είναι ανενεργός και ο άλλος ενεργός, και κάθε ένα δευτερόλεπτο, οι buffers ανταλλάσσονται, οπότε αυτός που ήταν ενεργός τώρα είναι ανενεργός και αυτός που ήταν ανενεργός τώρα είναι ενεργός. Στον ενεργό buffer γράφονται τα δεδομένα από το kernel τα οποία οποία με την ανταλλαγή των buffers στέλνονται και γίνονται διαθέσιμα προς διάβασμα από το userspace μέσω του πλέον ανενεργού buffer. Με άλλα λόγια, το kernel γράφει στον ενεργό buffer, και το userspace διαβάζει από τον ανενεργό. Αυτό έχει ως αποτέλεσμα την συνεχή ροή δεδομένων από το kernel προς το userspace, διότι ταυτόχρονα γίνονται διάβασμα και καταγραφή δεδομένων.

i++;

3.5 Scripts υψηλότερου επιπέδου

i++;

3.6 Παραδείγματα

i++;

4 kinst

Ο kinst είναι ένας νέος DTrace provider χαμηλού επιπέδου, ο οποίος αναπτύχθηκε από τον συντάκτη της παρούσας εργασίας, σε συνεργασία με τον Mark Johnston [8], ως μέρος του FreeBSD. Δημιουργήθηκε με στόχο να αντιμετωπίσει τους περιορισμούς του FBT (βλέπε ενότητα 3.1.1), δηλαδή την έλλειψη δυνατότητας παρακολούθησης inline συναρτήσεων, καθώς και γενικότερα της πιο εξειδικευμένης παρακολούθησης πέραν της αρχής ή/και του τέλους μίας συνάρτησης.

Εντός του πηγαίου κώδικα του FreeBSD, ο κώδικας του kinst βρίσκεται στο sys/cddl/dev/kinst [2] και υποστηρίζεται στις αρχιτεκτονικές x86-64 (AMD64), ARM64 και RISC-V.

Το όνομα "kinst" είναι εμπνευσμένο από την δημοσίευση των Tamches & Miller [3], στην οποία κάνουν αναφορά στο πειραματικό εργαλείο παρακολούθησης που ανέπτυξαν, ως "KernInst".

```
i++;
```

4.1 Χρήση

Ο kinst δέχεται τις παρακάτω τρεις συντάξεις:

- `kinst::<function>`:

Με αυτή την σύνταξη παρακολουθούνται όλες οι εντολές assembly της συνάρτησης `function`. Για παράδειγμα:

```
# dtrace -n 'kinst::vm_fault:'
dtrace: description 'kinst::vm_fault:' matched 1481 probes
CPU      ID                FUNCTION:NAME
  1  71225                vm_fault:0
  1  71226                vm_fault:1
  1  71227                vm_fault:4
  1  71228                vm_fault:6
  1  71229                vm_fault:8
  1  71230                vm_fault:10
  1  71231                vm_fault:12
  1  71232                vm_fault:13
  1  71233                vm_fault:20
[...]
```

Όπως φαίνεται στην έξοδο της παραπάνω εντολής, ο kinst έφτιαξε probes για 1481 εντολές εντός της συνάρτησης `vm_fault()`. Παρατηρώντας το disassembly της `vm_fault()`, βλέπουμε ότι το 1481 αντιστοιχεί στον συνολικό αριθμό εντολών της συνάρτησης.

Η εύρεση του disassembly μπορεί να γίνει με την χρήση ενός debugger, όπως του GDB [9]. Στα συγκεκριμένα παραδείγματα χρησιμοποιείται ο KGDB [10], μία παραλλαγή του GDB που παρέχεται από το FreeBSD, με μερικές επιπλέον λειτουργίες για την καλύτερη αποσφαλμάτωση του πυρήνα του:

```
# kgdb
(kgdb) set logging on
(kgdb) set pagination off
(kgdb) disas vm_fault
[...]
(kgdb) exit
$ grep 0x gdb.txt | wc -l
1481
```

- `kinst::<function>:<instruction>`

Με αυτή την σύνταξη παρακολουθείται εντός της συνάρτησης `function` μόνο η εντολή που βρίσκεται στο offset που ορίστηκε στο πεδίο `instruction`. Το offset αυτό μπορεί να βρεθεί στο disassembly της συνάρτησης. Για παράδειγμα:

```
# kgdb
(kgdb) disas /r vm_fault
Dump of assembler code for function vm_fault:
```

```

0xffffffff80f4e470 <+0>:    55      push    %rbp
0xffffffff80f4e471 <+1>:    48 89 e5      mov     %rsp,%rbp
0xffffffff80f4e474 <+4>:    41 57      push    %r15
[...]

```

Αν υποθεθεί ότι θέλουμε να παρακολουθήσουμε την τρίτη εντολή (`push %r15`) εντός της συνάρτησης `vm_fault()`, στην δεύτερη στήλη βλέπουμε ότι η εντολή αυτή βρίσκεται στο offset 4. Δίνοντας το offset αυτό στον `kinst`, μπορούμε να παρακολουθήσουμε μόνο αυτή την εντολή όποτε εκτελείται. Επιπλέον θα τυπώσουμε και τα περιεχόμενα του καταχωρητή RSI:

```

# dtrace -n 'kinst::vm_fault:4 {printf("%#x", regs[R_RSI]);}'
dtrace: description 'kinst::vm_fault:4 ' matched 1 probe
CPU      ID                      FUNCTION:NAME
  1  71225                      vm_fault:4 0x10652b411000
  1  71225                      vm_fault:4 0x10652c341000
  1  71225                      vm_fault:4 0x10652c342000
  1  71225                      vm_fault:4 0x10652c343000
[...]

```

- `kinst::<inline_function>:<entry|return>`

Με αυτή την σύνταξη παρακολουθείται η αρχή (`entry`) ή/και το τέλος (`return`) μίας *inline* συνάρτησης `inline_function`. Για παράδειγμα:

```

# dtrace -n 'kinst::critical_enter:return'
dtrace: description 'kinst::critical_enter:return' matched 130 probes
CPU      ID                      FUNCTION:NAME
  1  71024                      spinlock_enter:53
  0  71024                      spinlock_enter:53
  1  70992                      uma_zalloc_arg:49
  1  70925      malloc_type_zone_allocated:21
  1  70994                      uma_zfree_arg:365
  1  70924                      malloc_type_freed:21
  1  71024                      spinlock_enter:53
  0  71024                      spinlock_enter:53
  0  70947                      _epoch_enter_preempt:122
  0  70949                      _epoch_exit_preempt:28
[...]

```

```
i++;
```

4.2 Γιατί είναι χρήσιμο κάτι τέτοιο;

```
i++;
```

4.3 Ενορχήστρωση εντολών assembly

Οι πληροφορίες των probes στέλνονται από το `dtrace(1)` στο `kinst(4)` μέσω της `libdtrace`, χρησιμοποιώντας την συσκευή χαρακτήρων (character device) `/dev/dtrace/kinst`. Το `kinst(4)` είναι υπεύθυνο για τον εντοπισμό της συνάρτησης στο kernel, το `disassembly` της και την δημιουργία μίας εσωτερικής δομής δεδομένων με πληροφορίες σχετικά με την εντολή και το probe. Έπειτα δημιουργεί DTrace probes για κάθε μία από τις εντολές προς παρακολούθηση. Η εντολή προς παρακολούθηση αποθηκεύεται στην δομή δεδομένων

του probe για μετέπειτα χρήση και αντικαθιστάται με μία εντολή breakpoint. Μόλις ο CPU κατά την εκτέλεσή του, φτάσει στο breakpoint που έχει εγκατασταθεί από το DTrace, το kernel εισέρχεται στο DTrace μέσω του trap handler (`sys/$ARCH/$TARGET_ARCH/trap.c`) του kernel και εφόσον το breakpoint έχει εγκατασταθεί από τον `kinst` συγκεκριμένα, το kernel εν τέλει θα εισέλθει στην συνάρτηση `kinst_invop()`. Αυτή η συνάρτηση είναι υπεύθυνη για την παρακολούθηση της εντολής, είτε με την υλοποίηση της σε λογισμικό (emulation), είτε με την εκτέλεσή της σε «τραμπολίνo» (βλέπε ενότητα 2).

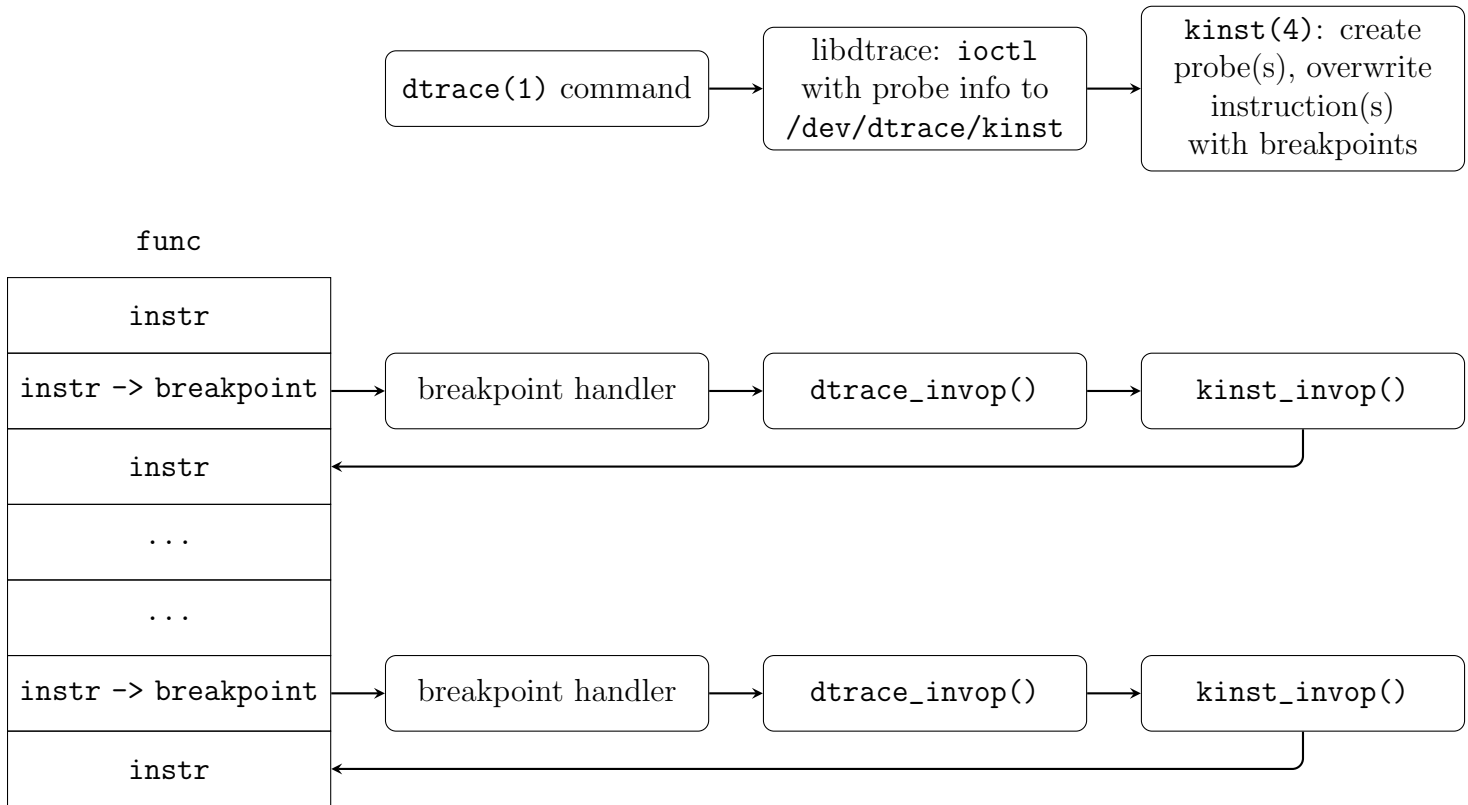


Figure 1: Γενική ροή.

`i++;`

4.4 «Τραμπολίνo»

Τραμπολίνo (trampoline) ονομάζεται ένα εκτελέσιμο κομμάτι μνήμης, το οποίο χρησιμοποιείται ως περιοχή αναπήδησης (jump) για την εκτέλεση κάποιας εντολής και επιστροφής στην κανονική ροή του προγράμματος.

Ο `kinst` κάνει χρήση τραμπολίνων για την εκτέλεση των εντολών των οποίων παρακολουθεί, σε αντίθεση με τον FBT (3.1.1), ο οποίος τις «μιμείται»/υλοποιεί σε λογισμικό (emulation). Ωστόσο, μία βασική διαφορά του FBT είναι ότι εφόσον έχει την δυνατότητα να παρακολουθήσει μόνο την αρχή και το τέλος μίας συνάρτησης, τα οποία σημεία ορίζονται με συγκεκριμένες εντολές assembly για κάθε αρχιτεκτονική¹, το σύνολο των εντολών που πρέπει να υλοποιήσει ο FBT είναι μικρό, καθώς και οι εντολές απλές, με αποτέλεσμα να μην χρειάζεται η επιπλέον πολυπλοκότητα που προσθέτει το τραμπολίνo. Ο `kinst` όμως μπορεί να παρακολουθήσει εν δυνάμει (σχεδόν) όλες τις εντολές των ISA (Instruction Set Architecture) των αρχιτεκτονικών που υποστηρίζει, οπότε είναι προφανές πως η προσέγγιση της υλοποίησης εντολών, δεν μπορεί να λειτουργήσει σε αυτήν την περίπτωση. Παρ' όλα αυτά, υπάρχουν μερικές εξαιρέσεις που θα συζητηθούν παρακάτω.

¹Για παράδειγμα, στην αρχιτεκτονική x86-64, η εντολή `push %rbp` δηλώνει το ξεκίνημα του προλόγου μίας συνάρτησης, και η εντολή `ret` το τέλος του επιλόγου.

Η χρήση του τραμπολίνου γίνεται ως εξής. Αρχικά, η εντολή προς παρακολούθηση αντιγράφεται στο τραμπολινό και αλλάζοντας την τιμή του PC (program counter), η εκτέλεση μεταφέρεται σε αυτό, όταν το kernel εισέλθει στον kinst με σκοπό την παρακολούθηση της συγκεκριμένης εντολής. Αφού εκτελεστεί η αντιγραμμένη εντολή εντός του τραμπολίνου, η εκτέλεση επιστρέφει στην επόμενη εντολή και το kernel επανέρχεται στην κανονική ροή εκτέλεσης.

i++;

4.4.1 Εσωτερική διάταξη

Ο kinst κρατάει μία ουρά TAILQ(9) από «κομμάτια» (chunks) μνήμης μεγέθους PAGE_SIZE, διαιρεμένα σε επιμέρους τραμπολίνια, χρησιμοποιώντας το BITSET(9).

Η μνήμη δεσμεύεται με την συνάρτηση vm_map_find(9) σε συνδυασμό με το όρισμα VM_PROT_EXECUTE, ώστε να επιτρέπεται η εκτέλεση εντολών εντός αυτών των θέσεων μνήμης. Όλες οι δεσμεύσεις γίνονται σε διευθύνσεις μεγαλύτερες της διεύθυνσης KERNBASE στην αρχιτεκτονική AMD64 και της VM_MIN_KERNEL_ADDRESS σε ARM64 και RISC-64 αντίστοιχα.

Όπως φαίνεται στο σχήμα 2, το τραμπολινό περιέχει την εντολή προς παρακολούθηση, ακολουθούμενη από την εντολή breakpoint στις αρχιτεκτονικές ARM64 και RISC-V και jmp αντίστοιχα στην AMD64. Η δεύτερη εντολή χρησιμοποιείται για την επιστροφή από το τραμπολινό και την συνέχιση της ροής εκτέλεσης (βλέπε ενότητες 4.4.2 και 4.4.3).

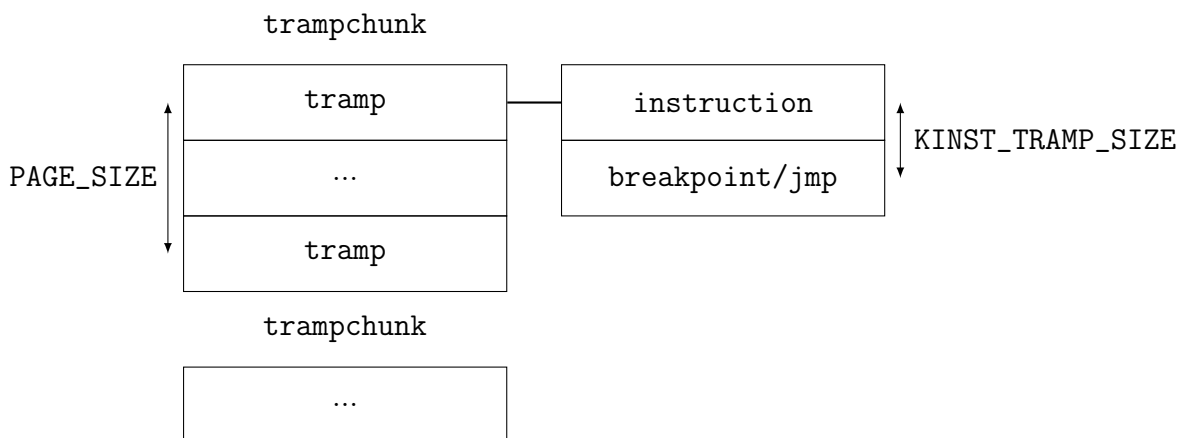


Figure 2: Διάταξη τραμπολίνου.

i++;

4.4.2 Σημειώσεις υλοποίησης για x86-64

Όπως αναφέρθηκε στην ενότητα 4.4.1, το τραμπολινό στην αρχιτεκτονική AMD64 περιέχει την εντολή προς παρακολούθηση, ακολουθούμενη από ένα far-jmpr στην επόμενη εντολή από αυτή που παρακολουθείται, προκειμένου να συνεχίσει η ροή εκτέλεσης.

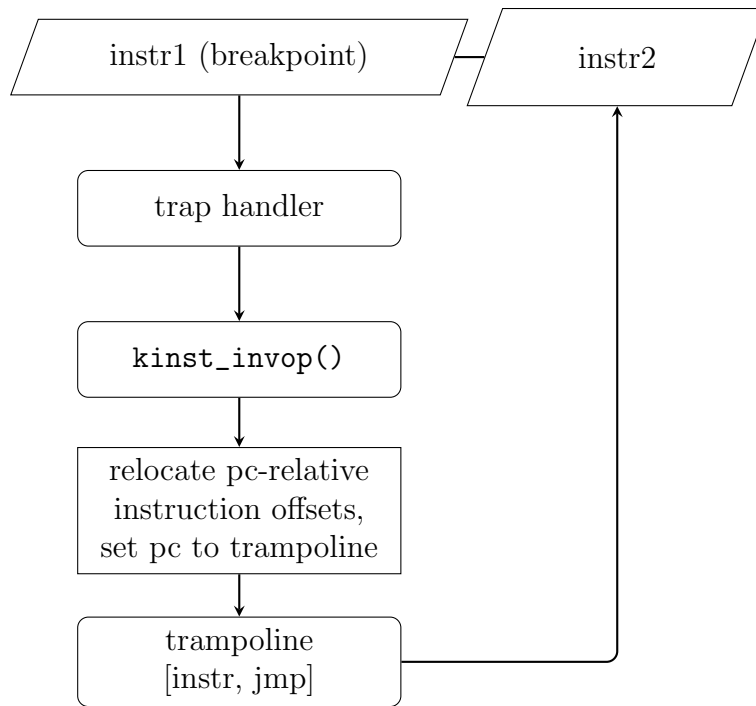


Figure 3: Διάγραμμα ροής amd64.

Για κάθε thread και CPU ορίζεται ένα τραμπολίνo, του οποίου τα περιεχόμενα αντικαθιστούνται σε κάθε $i++i$ (instrumentation). Ο λόγος για τον οποίο έχουμε τραμπολίνια και ανά thread, αλλά και ανά CPU, είναι ότι τα τραμπολίνια ανά thread δεν μπορούν να χρησιμοποιηθούν με ασφάλεια όταν το thread που εκτελείται είχε απενεργοποιημένα τα interrupts πριν το breakpoint. Αυτός ο μηχανισμός, αν και αποδοτικός από άποψη διαχείρισης μνήμης, μίας και ο αριθμός των τραμπολίνων είναι πάντα ίσος με τον αριθμό των threads και CPUs που έχει το σύστημα, έχει αποδειχθεί προβληματικός όταν τρέχει σε εικονικές μηχανές με πάνω από ένα vCPU, λόγω των συνεχών διαβασμάτων και επανεγγραφών του τραμπολίνου, κάθε φορά που το kernel εισέρχεται στην `kinst_invop()`. Γίνεται ήδη προσπάθεια ώστε να αντικατασταθούν τα τραμπολίνια ανά thread/CPU με τραμπολίνια ανά probe, όπως γίνεται στις αρχιτεκτονικές ARM64 και RISC-V.

Όταν μία εντολή αντιγράφεται στο τραμπολίνo, ο `kinst` έχει ειδικό χειρισμό για τις εντολές που κωδικοποιούνται offsets συναρτήσεως του καταχωρητή RIP (Instruction Pointer). Για τις εντολές αυτές το offset πρέπει να επανακωδικοποιηθεί, ώστε να είναι συνάρτηση της θέσης του τραμπολίνου, προκειμένου να μπορούν να εκτελεστούν εντός του τραμπολίνου.

Επιπλέον, η εντολή `call` πρέπει να υλοποιηθούν σε λογισμικό, διότι η επανακωδικοποίηση ενός offset ως προς το τραμπολίνo προϋποθέτει δέσμευση χώρου στο stack (βλέπε `bp_call` στο `sys/cddl/dev/dtrace/amd64/dtrace_asm.S`).

$i++i$

4.4.3 Σημειώσεις υλοποίησης για ARM64 και RISC-V

Σε αντίθεση με την αρχιτεκτονική AMD64, στις αρχιτεκτονικές ARM64 και RISC-V δεν είναι εφικτό να κωδικοποιηθεί far-jump σε μία μόνο εντολή, όπως και δεν είναι εφικτή η απευθείας αλλαγή τιμών καταχωρητών εντός του τραμπολίνου, με σκοπό την υλοποίηση του far-jump σε δύο εντολές. Μία εναλλακτική προσέγγιση για την επιστροφή από το τραμπολίνo σε αυτήν την περίπτωση, είναι η χρήση της εντολής breakpoint, αντί για far-jump.

Μόλις έχει εκτελεστεί η εντολή προς παρακολούθηση εντός του τραμπολίνου και ο CPU φτάσει στην εντολή breakpoint, το kernel θα αναπηδήσει πίσω στον trap handler και θα επανεισέλθει στην `kinst_invop()` (βλέπε ενότητα 4.3). Ο `kinst` κρατάει μία εσωτερική ανά CPU δομή προκειμένου να διαφοροποιεί μεταξύ

breakpoints που προκλήθηκαν από το τραμπολίνο, ή από probe που ενεργοποιήθηκε. Στην πρώτη περίπτωση, η `kinst_invop()` αποθηκεύει τον τρέχον καταχωρητή κατάστασης (state register), απενεργοποιεί τα interrupts ώστε να μην μπορεί να διακοπεί το thread κατά την διάρκεια της εκτέλεσης του «διπλού breakpoint», καθώς και ορίζει τον program counter στην διεύθυνση του τραμπολίνου. Στην δεύτερη περίπτωση, δηλαδή όταν έχουμε επιστρέψει από το τραμπολίνο και πρέπει να συνεχίσει η ροή εκτέλεσης, η `kinst_invop()` επαναφέρει τον καταχωρητή κατάστασης και ενεργοποιεί τα interrupts και χειροκίνητα ορίζει τον program counter στην επόμενη εντολή από αυτή που μόλις παρακολούθηθηκε. Το σχήμα 4 δείχνει το διάγραμμα ροής αυτού του μηχανισμού.

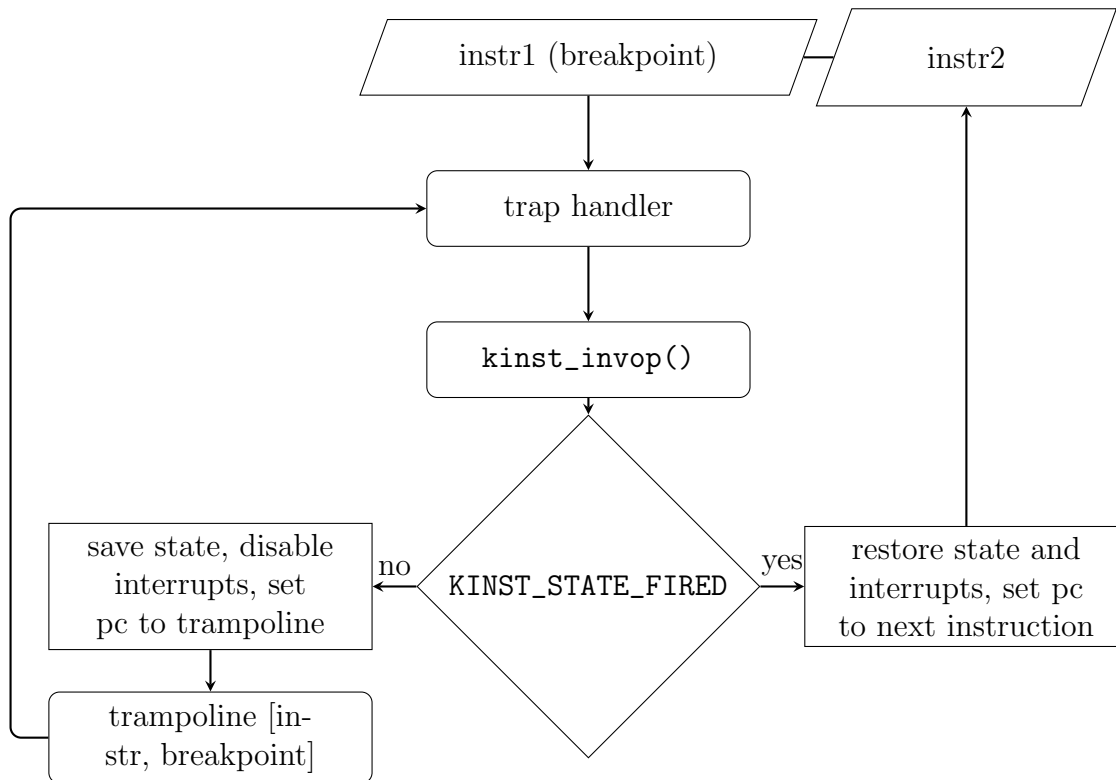


Figure 4: Διάγραμμα ροής arm64 & riscv.

Κάθε probe έχει το δικό της τραμπολίνο, το οποίο γράφεται κατά την διάρκεια ανάλυσης της εντολής, το οποίο σε αντίθεση με την αρχιτεκτονική AMD64, αποφεύγει την ανάγκη για διάβασμα και επανεγγραφή τραμπολίνων ανά thread/CPU κάθε φορά που εκτελείται η `kinst_invop()`.

`i++;`

4.5 Παρακολούθηση inline συναρτήσεων

Η παρακολούθηση inline συναρτήσεων στον `kinst` δεν είναι τίποτα παραπάνω από ένα συντακτικό στρώμα που έχει προστεθεί στην `libdtrace`. Όταν ο χρήστης ζητάει ένα probe της μορφής `kinst::<function>:<entry|return>`², η `libdtrace` αναλύει τις πληροφορίες DWARF (βλέπε ενότητα 4.5.4) όλων των φορτωμένων kernel modules, με σκοπό να βρει αν η συνάρτηση `<function>` είναι inline. Σε αυτή την περίπτωση, δημιουργεί `kinst` probes στις συναρτήσεις και ακριβή offset(s), στα οποία έχει βρεθεί inline αντίγραφο της `<function>`. Ωστόσο, αν η `<function>` δεν είναι inline, το probe μετατρέπεται αυτόματα σε FBT probe, μίας ο FBT είναι

²Σε αντίθεση με τον FBT, ο `kinst` δεν μπορεί να δεχθεί άδειο το πεδίο probe όταν πρόκειται για παρακολούθηση inline συναρτήσεων. Προκειμένου να μπορέσει να ενεργοποιηθεί ο κώδικας εντοπισμού inline συναρτήσεων εντός του `libdtrace`, πρέπει αναγκαστικά να οριστεί είτε η λέξη `entry` είτε `return`.

ιδανικός για την παρακολούθηση της αρχής και τέλους μη-inline συναρτήσεων. Το σχήμα 5 παρουσιάζει ένα παράδειγμα του πώς η libdtrace χειρίζεται inline και μη-inline συναρτήσεις.

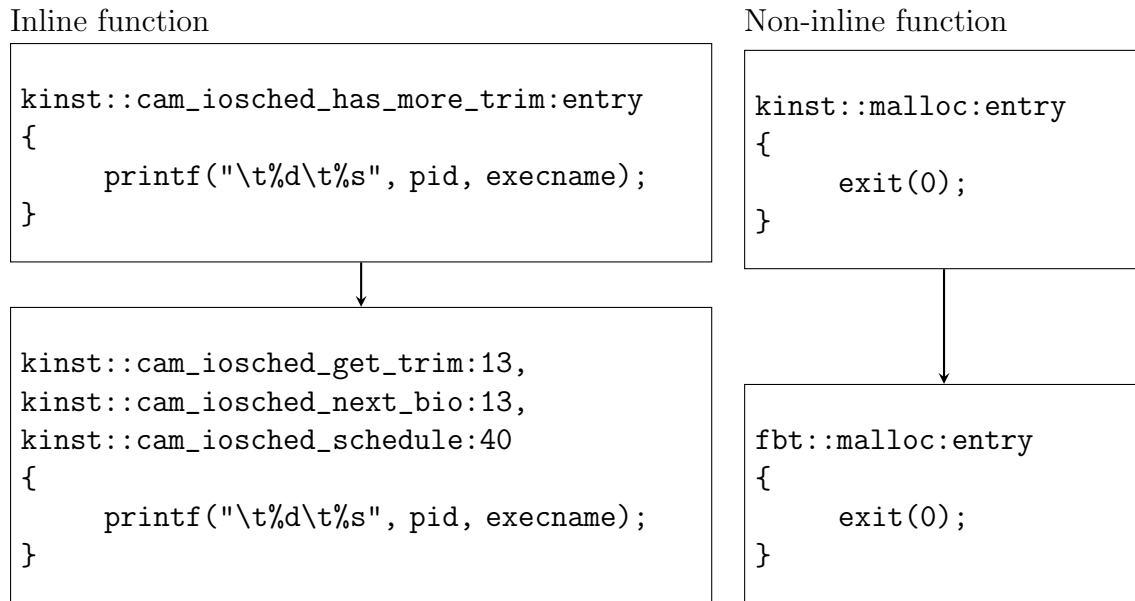


Figure 5: Συντακτικοί μετασχηματισμοί υλοποιημένοι στο `cddl/contrib/opensolaris/lib/libdtrace/common/dt_sugar.c`

Υπάρχει περίπτωση να υπάρχουν inline αλλά και μη-inline ορισμοί της ίδιας συνάρτησης. Σε αυτήν την περίπτωση, η libdtrace, πέρα από τα kinst probes για τα inline αντίγραφα, δημιουργεί και ένα επιπλέον FBT probe για την μη-inline συνάρτηση.

Για την παραγωγή των εξόδων που παρουσιάζονται στο σχήμα 5, χρησιμοποιείται η επιλογή `-d` στην εντολή `dtrace(1)`, η οποία τυπώνει το D script αφού έχουν εφαρμοστεί όλοι οι συντακτικοί μετασχηματισμοί. Η επιλογή αυτή προστέθηκε με το commit `1e136a9cbd3a`.

`i++i`

4.5.1 Τι είναι inline συναρτήσεις

`i++i`

4.5.2 Γιατί είναι δύσκολη η παρακολούθηση inline συναρτήσεων

`i++i`

4.5.3 Πως λύνει ο kinst αυτό το πρόβλημα

Στην πραγματικότητα, ο kinst δεν διαθέτει επιπλέον λογική όσον αφορά τις inline συναρτήσεις — η λύση αυτού του προβλήματος βρίσκεται σε προηγούμενο στάδιο, αυτό της συντακτικής ανάλυσης του D script από την βιβλιοθήκη DTrace (libdtrace). `i++i`

`i++i`

4.5.4 Πρότυπο DWARF

Το πρότυπο αποσφαλμάτωσης DWARF [4] είναι ένα πρότυπο το οποίο χρησιμοποιείται από μεταγλωττιστές και προγράμματα αποσφαλμάτωσης (debuggers) για την κωδικοποίηση χρήσιμων πληροφοριών, όπως ονόματα συναρτήσεων ή θέσεις μεταβλητών, και πολλών άλλων, για κάθε μονάδα μεταγλώττισης (compilation unit).

This document defines a format for describing programs to facilitate user source level debugging. This description can be generated by compilers, assemblers and linkage editors. It can be used by debuggers and other tools.

– The DWARF Standard: <https://dwarfstd.org/>

Οι πληροφορίες αυτές αναπαριστώνται ως ένα δέντρο από εγγραφές, οι οποίες ονομάζονται DIEs (Debugging Information Entries). Η κάθε μονάδα μεταγλώττισης αντιστοιχεί σε ένα τέτοιο δέντρο. Για καλύτερη κατανόηση, ας θεωρήσουμε το παρακάτω θεωρητικό δέντρο:

```
i++i
```

Κατά την μεταγλώττιση ενός C προγράμματος, η ενσωμάτωση των πληροφοριών DWARF εντός του μεταγλωττισμένου αρχείου μπορεί να γίνει με την χρήση της επιλογής `-g` (παραγωγή πληροφοριών αποσφαλμάτωσης):

```
cc <file>.c -g -o <file>
```

Οι πληροφορίες DWARF μπορούν έπειτα να διαβαστούν είτε με το πρόγραμμα `readelf` (με την επιλογή `-wi`):

```
readelf -wi <file>
```

Είτε με το `dwarfdump`:

```
dwarfdump <file>
```

Οι δηλώσεις των inline συναρτήσεων στο πρότυπο DWARF αναπαριστώνται με την ετικέτα `DW_TAG_subprogram`, σε συνδυασμό με την ιδιότητα `DW_AT_inline`. Ωστόσο για κάθε δήλωση μίας inline συνάρτησης, υπάρχουν και αντίγραφά της, ένα για κάθε σημείο στο οποίο η συνάρτηση έχει μετατραπεί σε inline εντός μίας άλλης συνάρτησης. Τα αντίγραφα αυτά αναπαριστώνται με την ετικέτα `DW_TAG_inlined_subroutine` και μπορούν να περιέχουν τις παρακάτω ιδιότητες:

- `DW_AT_abstract_origin`: Offset του DIE σε σχέση με την δήλωση της inline συνάρτησης.
- `DW_AT_call_file`: Ακέραιος αριθμός που δηλώνει το αρχείο στο οποίο καλείται η συνάρτηση.
- `DW_AT_call_line`: Γραμμή κώδικα εντός του αρχείου, στο οποίο καλείται η συνάρτηση.
- `DW_AT_call_column`: Στήλη εντός της γραμμής, όπου ξεκινάει η κλήση της συνάρτησης.
- `DW_AT_low_pc` και `DW_AT_high_pc`: Κάτω και άνω όρια (διευθύνσεις μνήμης) εντός των οποίων βρίσκεται το αντίγραφο.
- `DW_AT_ranges`: Σειρές ορίων εντός των οποίων βρίσκεται το αντίγραφο, σε περίπτωση που το αντίγραφο δεν βρίσκεται σε ενιαίες θέσεις μνήμης.

Όσον αφορά τις inline συναρτήσεις, ας υποθέσουμε την παρακάτω εντολή DTrace:

```
# dtrace -n 'kinst::vfs_freevnodes_dec:entry'
```

Εφόσον η συνάρτηση `vfs_freevnodes_dec()` είναι μέρος του FreeBSD kernel, η `libdtrace` πρέπει αρχικά να αναλύσει τα δεδομένα DWARF του kernel, τα οποία βρίσκονται στο αρχείο `/usr/lib/debug/boot/kernel/kernel.debug`, και να βρει αν η `vfs_freevnodes_dec()` είναι inline. Αν και όλα αυτά γίνονται εντός της `libdtrace`, χάριν κατανόησης και απεικόνισης της διαδικασίας που ακολουθείται, θα χρησιμοποιήσουμε την παρακάτω εντολή για να εμφανίσουμε τα δεδομένα DWARF του kernel:

```
readelf -wi /usr/lib/debug/boot/kernel/kernel.debug
```

Και έπειτα θα ψάξουμε για το παρακάτω DIE:

```

<1><1dfa144>: Abbrev Number: 94 (DW_TAG_subprogram)
  <1dfa145>   DW_AT_name           : (indirect string) vfs_freevnodes_dec
  <1dfa149>   DW_AT_decl_file      : 1
  <1dfa14a>   DW_AT_decl_line     : 1447
  <1dfa14c>   DW_AT_prototyped    : 1
  <1dfa14c>   DW_AT_inline        : 1

```

Listing 1: DIE δήλωσης inline συνάρτησης

Η ετικέτα `DW_TAG_subprogram` υποδηλώνει ότι το DIE αναφέρεται σε συνάρτηση, και η ιδιότητα `DW_AT_inline` ότι η συνάρτηση είναι inlined. Τώρα που η `libdtrace` ξέρει ότι η συνάρτηση είναι πράγματι inlined, πρέπει να βρει τα DIE που αντιστοιχούν στο κάθε inline αντίγραφο, όπως το παρακάτω:

```

<3><1dfe45e>: Abbrev Number: 24 (DW_TAG_inlined_subroutine)
  <1dfe45f>   DW_AT_abstract_origin: <0x1dfa144>
  <1dfe463>   DW_AT_low_pc         : 0xffffffff80cf701d
  <1dfe46b>   DW_AT_high_pc        : 0x38
  <1dfe46f>   DW_AT_call_file      : 1
  <1dfe470>   DW_AT_call_line     : 3458
  <1dfe472>   DW_AT_call_column   : 5

```

Listing 2: DIE αντιγράφου inline συνάρτησης

Όπως αναφέρθηκε παραπάνω, τα inline αντίγραφα μίας συνάρτησης υποδηλώνονται με την ετικέτα `DW_TAG_inlined_subroutine`. Η ιδιότητα `DW_AT_abstract_origin` ορίζει το offset στο οποίο βρίσκεται η δήλωση της δήλωσης της συνάρτησης, σε αυτή την περίπτωση `0x1dfa144`, το οποίο αντιστοιχεί στο DIE της δήλωσης της `vfs_freevnodes_dec()` (βλέπε Listing 1).

```

i++;
```

4.5.5 Πρότυπο ELF

```

i++;
```

4.5.6 Υπολογισμός ορίων κλήσεων

Το DWARF καθορίζει τα όρια κλήσεων με έναν από τους δύο τρόπους:

- Ορίζοντας τις ιδιότητες `DW_AT_low_pc` και `DW_AT_high_pc`.
- Ορίζοντας την ιδιότητα `DW_AT_ranges` όταν το εύρος διευθύνσεων του inline αντιγράφου δεν είναι συνεχόμενο.

Στην πρώτη περίπτωση, το κάτω όριο της θέσης του inline αντιγράφου ορίζεται στο `DW_AT_low_pc` και το άνω όριο υπολογίζεται προσθέτοντας το `DW_AT_high_pc` στο `DW_AT_low_pc`. Χρησιμοποιώντας ως παράδειγμα την συνάρτηση `vfs_freevnodes_dec()`, ως συνέχεια της ενότητας 4.5.4:

```

<3><1dfe45e>: Abbrev Number: 24 (DW_TAG_inlined_subroutine)
  <1dfe45f>   DW_AT_abstract_origin: <0x1dfa144>
  <1dfe463>   DW_AT_low_pc         : 0xffffffff80cf701d
  <1dfe46b>   DW_AT_high_pc        : 0x38
  <1dfe46f>   DW_AT_call_file      : 1
  <1dfe470>   DW_AT_call_line     : 3458
  <1dfe472>   DW_AT_call_column   : 5

```

Listing 3: DIE με κάτω και άνω όρια PC

Χρησιμοποιώντας τον τύπο που αναφέρθηκε στην προηγούμενη παράγραφο, έχουμε τα παρακάτω όρια:

$$\begin{aligned} Bound_{low} &= PC_{low} = 0xffffffff80cf701d \\ Bound_{high} &= PC_{low} + PC_{high} = 0xffffffff80cf701d + 0x38 = \\ &0xffffffff80cf7055 \end{aligned} \quad (1)$$

Η δεύτερη περίπτωση είναι λίγο πιο περίπλοκη. Ας υποθέσουμε το παρακάτω DIE ενός inline αντιγράφου της `vfs_freevnodes_dec()` που έχει την ιδιότητα `DW_AT_ranges`:

```
<3><1dfd2e2>: Abbrev Number: 58 (DW_TAG_inlined_subroutine)
<1dfd2e3>   DW_AT_abstract_origin: <0x1dfa144>
<1dfd2e7>   DW_AT_ranges          : 0x1f1290
<1dfd2eb>   DW_AT_call_file       : 1
<1dfd2ec>   DW_AT_call_line      : 3405
<1dfd2ee>   DW_AT_call_column    : 3
```

Listing 4: DIE με όρια `DW_AT_ranges`

Η ιδιότητα `DW_AT_ranges` αναφέρεται στην ενότητα `.debug_ranges` που βρίσκεται εντός των debug αρχείων και μπορεί να εμφανισθεί τρέχοντας την εντολή `dwarfdump -N <file>`. Αναζητώντας για την παραπάνω τιμή `DW_AT_ranges 0x1f1290`, βρίσκουμε την αντίστοιχη ομάδα εύρους:

Ranges group 38809:

```
ranges: 3 at .debug_ranges offset 2036368 (0x001f1290) (48 bytes)
[ 0] range entry    0x000025c8 0x000025f9
[ 1] range entry    0x0000261a 0x00002621
[ 2] range end      0x00000000 0x00000000
```

Listing 5: Παράδειγμα `DW_AT_ranges`

Όλες οι γραμμές «range entry» αντιστοιχούν σε διαφορετικά εύρη διευθύνσεων στα οποία έχει χωριστεί το inline αντίγραφο, συνήθως ως αποτέλεσμα πρόωρων return. Τα όρια κλήσεων υπολογίζονται προσθέτοντας κάθε τιμή των «range entries» στο κάτω όριο (`DW_AT_low_pc`) του root DIE, δηλαδή του DIE ανώτατου επιπέδου, που αντιστοιχεί στο αρχείο εντός του οποίου είναι υλοποιημένη η inline συνάρτηση:

```
<0><1dee9fb>: Abbrev Number: 1 (DW_TAG_compile_unit)
<1dee9fc>   DW_AT_producer        : (indirect string) FreeBSD clang version 13.0.0 (
git@github.com:llvm/llvm-project.git llvmorg-13.0.0-0-gd7b669b3a303)
<1deea00>   DW_AT_language        : 12 (C99)
<1deea02>   DW_AT_name            : (indirect string) /usr/src/sys/kern/vfs_subr.c
<1deea06>   DW_AT_stmt_list       : 0x6cb448
<1deea0a>   DW_AT_comp_dir        : (indirect string) /usr/obj/usr/src/amd64.amd64/sys/
GENERIC
<1deea0e>   DW_AT_low_pc          : 0xffffffff80cf4020
<1deea16>   DW_AT_high_pc         : 0xde3d
```

Listing 6: DIE μονάδας μεταγλώττισης

Προσθέτοντας τα range entries στο κάτω όριο του root DIE (`0xffffffff80cf4020`), έχουμε τα παρακάτω όρια κλήσης του inline αντιγράφου:

Πρώτο range entry:

$$\begin{aligned} Bound_{low} &= Root_{lowpc} + Range_{low} = 0xffffffff80cf4020 + 0x000025c8 = \\ &0xffffffff80cf65e8 \\ Bound_{high} &= Root_{lowpc} + Range_{high} = 0xffffffff80cf4020 + 0x000025f9 = \\ &0xffffffff80cf6619 \end{aligned} \quad (2)$$

Δεύτερο range entry:

$$\begin{aligned}
 Bound1_{low} &= Root_{lowpc} + Range1_{low} = 0xffffffff80cf4020 + 0x0000261a = \\
 &0xffffffff80cf663a \\
 Bound1_{high} &= Root_{lowpc} + Range1_{high} = 0xffffffff80cf4020 + 0x00002621 = \\
 &0xffffffff80cf6641
 \end{aligned} \tag{3}$$

Το $Bound0_{low}$ είναι το σημείο `entry`, και τα $Bound0_{high}$ και $Bound1_{high}$ τα σημεία `return`.
i++;

4.5.7 Εύρεση καλούσας συνάρτησης

Αφού έχουν υπολογισθεί τα όρια κλήσεων των inline αντιγράφων, η libdtrace μπορεί να προχωρήσει στην εύρεση του ονόματος των καλούντων συναρτήσεων αυτών των αντιγράφων. Τα ονόματα αυτά χρειάζονται για την δημιουργία `kinst probes`. Αυτό γίνεται βρίσκοντας εντός ποιού συμβόλου ELF βρίσκονται τα όρια κλήσεων, όπως εκφράζεται από την παρακάτω συνθήκη:

$$Sym_{low} \leq Inline_{low} \leq Inline_{high} \leq Sym_{high}$$

i++;

4.5.8 Υπολογισμός των offsets entry και return

Αυτό είναι το τελευταίο βήμα που πρέπει να ακολουθήσει η libdtrace προκειμένου να μετατρέψει το probe inline συνάρτησης σε `kinst probe`. Εδώ βρίσκει τα όρια των καλούντων συναρτήσεων μέσω των πληροφοριών ELF και, χρησιμοποιώντας τα όρια των inline αντιγράφων, υπολογίζει τα ακριβή `entry` ή `return offsets`.

Θεωρητικά, τα `entry` και `return offsets` θα υπολογίζονταν απλώς ως εξής:

$$Entry = Inline_{low} - Caller_{low}$$

$$Return = Inline_{high} - Caller_{low}$$

Ωστόσο, προκύπτει ότι υπάρχουν πολλαπλά προβλήματα που περιπλέκουν τους υπολογισμούς.

i++;

Τέλος, τα offsets που υπολογίστηκαν χρησιμοποιούνται για την δημιουργία κανονικών `kinst probes` της μορφής `kinst::<function>:<instruction>`:

```
# dtrace -dn 'kinst::cam_iosched_has_more_trim:entry'
kinst::cam_iosched_get_trim:13,
kinst::cam_iosched_next_bio:13,
kinst::cam_iosched_schedule:40
{
}
```

dtrace: description 'kinst::cam_iosched_has_more_trim:entry' matched 4 probes

CPU	ID	FUNCTION:NAME
0	81502	cam_iosched_schedule:40
0	81501	cam_iosched_next_bio:13
2	81502	cam_iosched_schedule:40
1	81502	cam_iosched_next_bio:13
1	81503	cam_iosched_schedule:40

~c

Listing 7: Μετατροπή inline probe σε κανονικό

5 Πειράματα

i++i

6 Συμπεράσματα

i++i

7 Βιβλιογραφία

- [1] Illumos Operating System “Dynamic Tracing Guide”. <https://illumos.org/books/dtrace>
- [2] FreeBSD src “kinst” <https://cgит.freebsd.org/src/tree/sys/cddl/dev/kinst>
- [3] Tamches, Ariel & Miller, Barton P. “Fine-Grained Dynamic Instrumentation of Commodity Operating System Kernels”. https://www.usenix.org/legacy/publications/library/proceedings/osdi99/full_papers/tamches/
- [4] The DWARF Standard. <https://dwarfstd.org/>
- [5] Christos Margiolis “Using DWARF to find call sites of inline functions”. https://margiolis.net/w/dwarf_inline/
- [6] Christos Margiolis “Inline function tracing with the kinst DTrace provider”. https://margiolis.net/w/kinst_inline/
- [7] Sourcehut, inlinecall(1). <https://github.com/christosmarg/inlinecall>
- [8] Mark Johnston “Introduction to DTrace on FreeBSD”. <https://www.youtube.com/watch?v=E06GVdH-LX0>
- [9] GNU “GDB: The GNU Project Debugger”. <https://www.sourceware.org/gdb/>
- [10] FreeBSD manual pages “kgdb(1)”. [https://man.freebsd.org/cgi/man.cgi?query=kgdb\(1\)](https://man.freebsd.org/cgi/man.cgi?query=kgdb(1))

8 Παράρτημα

i++i